

Application Web de gestion d'un portefeuille d'investissement

Blazor Server, Entity Framework Core, architecture en couches

Équipe: Mohamed Jegham et Louay Ben Mansour

Résumé

Ce document présente la conception et la réalisation d'une application Web complète de *gestion de portefeuille* (actifs financiers, transactions, budget, tableau de bord analytique). L'implémentation repose sur **.NET 8**, **Blazor Web App** (mode interactif serveur), **Entity Framework Core** en approche *Code First* avec base **SQLite**, et une architecture découpée en couches (**Core**, **Infrastructure**, **Services**, **Web**). Une extension de type *data science* complète les indicateurs classiques : volatilité des rendements, moyenne mobile (SMA), ratio de Sharpe simplifié, drawdown maximal, backtest de règle de tendance, alertes et graphiques (*Chart.js*). Le rapport détaille les exigences pédagogiques, le modèle de données, les services métier et les pistes d'évolution.

Table des matières

1	Introduction	3
1.1	Contexte	3
1.2	Objectifs du projet	3
1.3	Périmètre technique	3
1.4	Organisation du rapport	4
2	Exigences et fonctionnalités	5
2.1	Exigences pédagogiques (rappel)	5
2.2	Fonctionnalités métier couvertes	5
2.3	Extension « créativité »	5
3	Architecture logicielle	7
3.1	Vue en couches	7
3.2	Principes appliqués	7
3.3	Flux typique d'une requête utilisateur	7
3.4	Diagramme de classes	8
4	Modèle de données et persistance	9
4.1	Entités principales	9
4.2	Relations et intégrité	9
4.3	Migrations	9
4.4	Choix SQLite	9
5	Réalisation technique	10
5.1	Composition racine et pipeline	10
5.2	Services métier	10
5.2.1	Portefeuille (PortfolioService)	10
5.2.2	Transactions (TransactionService)	10
5.2.3	Actifs (AssetService)	10
5.2.4	Prix marché (PriceDataService)	10
5.2.5	Alertes (PortfolioAlertService)	11
5.3	Validation	11
5.4	Asynchronisme	11
6	Extension analytique et data science	12
6.1	Données d'entrée	12
6.2	Moyenne mobile simple (SMA)	12
6.3	Volatilité	12
6.4	Ratio de Sharpe simplifié	12
6.5	Drawdown maximal	13
6.6	Backtest de règle SMA	13
6.7	Intérêt pour la note « créativité »	13

7	Interface utilisateur (Blazor)	14
7.1	Structure de navigation	14
7.2	Pages principales	14
7.3	Technologies front	14
7.4	Mode de rendu	14
8	Validation et déploiement local	15
8.1	Vérifications automatisées	15
8.2	Vérifications manuelles recommandées	15
8.3	Limites connues	15
9	Conclusion et perspectives	16
9.1	Bilan	16
9.2	Pistes d'évolution	16
9.3	Remerciements / répartition travail d'équipe	16
A	Annexe A — Extraits de code significatifs	17
A.1	Point d'entrée et migrations (Program.cs)	17
A.2	Injection de dépendances (AddPortfolioServices)	17
A.3	Lecture complémentaire	18
B	Annexe B — Commandes utiles (.NET CLI)	19
B.1	Restaurer et compiler	19
B.2	Exécuter l'application Web	19
B.3	Migrations Entity Framework Core	19
B.4	Problème de fichiers verrouillés (Windows)	19
B.5	Compiler le PDF du rapport	20

Chapitre 1

Introduction

1.1 Contexte

La gestion d'un portefeuille d'investissement (actions, crypto-actifs) nécessite le suivi des positions, des liquidités, des opérations d'achat et de vente, ainsi que des indicateurs de performance. Dans le cadre d'un cours sur le développement d'applications Web avec *.NET*, le présent projet met en œuvre une solution complète répondant à un cahier des charges imposant notamment Blazor, Entity Framework Core, une architecture propre, un tableau de bord analytique et l'utilisation de l'asynchronisme.

1.2 Objectifs du projet

Les objectifs principaux sont les suivants :

- proposer une interface Web permettant la **gestion des actifs** et la **saisie des transactions** (achat / vente) avec validation ;
- suivre un **budget d'investissement** et l'**historique des opérations** ;
- afficher un **tableau de bord** avec valeur du portefeuille, gain / perte, répartition par actif et par type, et des graphiques ;
- intégrer une **extension analytique** (créativité) : indicateurs statistiques sur l'historique de prix, aide à la décision, backtest simple ;
- respecter une **architecture en couches** et des bonnes pratiques (séparation UI / services / accès données).

1.3 Périmètre technique

Le dépôt logiciel est organisé en plusieurs projets au sein d'une solution **.slnx** :

GestionPortefeuille.Core Entités, énumérations, modèles de projection (DTO), interfaces de services, options de configuration.

GestionPortefeuille.Infrastructure DbContext, migrations EF Core, initialisation des données (*seed*).

GestionPortefeuille.Services Implémentations des services (portefeuille, transactions, actifs, analytique, alertes, prix marché).

GestionPortefeuille.Web Application Blazor, pages, composants, fichiers statiques, configuration.

La base de données relationnelle est **SQLite** (fichier **portfolio.db**), choix adapté à un rendu académique (portabilité, déploiement simple).

1.4 Organisation du rapport

Le chapitre 2 rappelle les exigences du cours et la cartographie fonctionnelle. Le chapitre 3 décrit l'architecture logicielle. Le chapitre 4 présente le modèle de données et les migrations. Le chapitre 5 détaille la réalisation (services, asynchronisme, validation). Le chapitre 6 expose l'extension *data science*. Le chapitre 7 concerne l'interface utilisateur. Le chapitre 8 résume la validation. Les annexes fournissent des extraits de code et les commandes utiles.

Chapitre 2

Exigences et fonctionnalités

2.1 Exigences pédagogiques (rappel)

Le projet vise à satisfaire les critères suivants, tels que formulés dans l'énoncé du cours :

- **Blazor Web Application** : utilisation de Blazor en mode interactif serveur (`InteractiveServer`) pour les pages nécessitant des interactions riches (formulaires, graphiques, thème).
- **Architecture propre** : séparation des responsabilités entre UI, services métier et persistance.
- **Entity Framework Core** : approche *Code First* avec **migrations** versionnées.
- **CRUD** complet sur les entités principales (actifs, transactions ; budget ajustable).
- **Recherche et filtrage** dynamiques via LINQ (actifs : texte, type ; transactions : pagination et tri).
- **Tableau de bord analytique** : indicateurs, graphiques (répartition, courbe de valeur dans le temps).
- **Validation** : *Data Annotations* sur les entités et formulaires Blazor.
- **Asynchronisme** : méthodes `async/await` côté services et accès EF (`ToListAsync`, `SaveChangesAsync`, etc.).

2.2 Fonctionnalités métier couvertes

2.3 Extension « créativité »

L'extension retenue apporte une **valeur analytique et décisionnelle** complémentaire :

- quantification du **risque** (volatilité, drawdown) ;
- **tendance** par rapport à une moyenne mobile ;
- **comparaison risque/rendement** via un indicateur de type Sharpe simplifié ;
- **simulation historique** d'une règle d'investissement (backtest) pour argumenter à l'oral.

Cette extension reste cohérente avec le domaine « portefeuille » et s'appuie sur un historique de prix (généré si absent lors du premier chargement analytique).

TABLE 2.1 – Fonctionnalités par domaine

Domaine	Description
Actifs	CRUD, symbole unique, type Stock/Crypto, prix courant ; liste paginée, tri, recherche avec temporisation (<i>debounce</i>).
Transactions	Saisie achat/vente, calcul du montant total, contrôle de liquidité et de quantité détenue ; historique paginé ; export CSV.
Budget	Consultation et mise à jour du budget initial et de la liquidité disponible.
Portefeuille	Instantané : valeur titres, cash, valeur totale, gain/perte vs budget initial, ROI, répartition par actif et par type.
Historique valeur	Points journaliers (<code>PortfolioValuePoint</code>) mis à jour à la visite du tableau de bord et après transactions.
Marchés (optionnel)	Mode <i>simulé</i> , mode <i>API</i> (Finnhub + CoinGecko avec cache), import CSV des prix.
Alertes	Seuils configurables : perte vs budget, liquidité basse, volatilité élevée par actif.
Data science	SMA, volatilité annualisée approximative, Sharpe-like, draw-down, signal tendance, backtest SMA.
UX	Thème clair/sombre (Bootstrap), graphiques Chart.js, navigation par menu.

Chapitre 3

Architecture logicielle

3.1 Vue en couches

L'application suit une architecture en **quatre projets** alignés sur une *clean architecture* simplifiée :

1. **Core** : aucune dépendance vers Infrastructure ni Web ; contient le modèle conceptuel (entités, enums), les contrats (**interface**) et les DTO utilisés par l'UI ou les services.
2. **Infrastructure** : dépend de Core ; implémente la persistance (EF Core, SQLite), le **DbContext** et les migrations.
3. **Services** : dépend de Core et Infrastructure ; concentre la logique métier, le calcul du portefeuille, les règles de transaction, l'analytique et l'intégration HTTP pour les cours.
4. **Web** : point d'entrée ASP.NET Core ; configure l'injection de dépendances, le pipeline HTTP, Blazor et référence les trois autres couches.

3.2 Principes appliqués

Dépendances vers l'intérieur L'UI et le composition root (**Program.cs**) résolvent les interfaces vers des implémentations concrètes enregistrées dans le conteneur IoC.

Testabilité Les services prennent en charge des abstractions (**DbContext**, **HttpClient** typé, **IMemoryCache**, **IOptions**) injectables.

Cohésion Un fichier **ServiceCollectionExtensions** centralise l'enregistrement des services métier et la configuration des options (**Alerts**, **PriceData**).

3.3 Flux typique d'une requête utilisateur

Pour une action sur le tableau de bord :

1. le composant Blazor (mode serveur) appelle un service injecté (**IPortfolioService**, **IAalyticsService**, **IPortfolioAlertService**) ;
2. le service interroge **AppDbContext** de manière asynchrone ;
3. les résultats sont projetés en modèles (**PortfolioSnapshot**, **AssetAnalytics**, **PortfolioAlert**) ;
4. l'UI met à jour l'état et, le cas échéant, invoque du JavaScript (**Chart.js**) pour le rendu graphique.

3.4 Diagramme de classes

Un diagramme de classes Mermaid est fourni dans le dépôt à la racine : `DiagrammeClasses.md`. Il est recommandé d'**exporter** les figures (PNG/SVG) depuis <https://mermaid.live> et de les insérer ici avec :

```
\begin{figure}[h]
  \centering
  \includegraphics[width=\textwidth]{figures/domaine-ef.png}
  \caption{Modèle entités-associations (extrait).}
\end{figure}
```

Créer un dossier `Rapport/figures/` et y placer les exports pour que la compilation $\text{\LaTeX}$ réussisse.

Chapitre 4

Modèle de données et persistance

4.1 Entités principales

Asset Représente un instrument : symbole (unique), nom, type (`Stock` / `Crypto`), prix courant. Collections de navigation vers les transactions et l'historique de prix.

Transaction Mouvement d'achat ou de vente : quantité, prix unitaire, date, montant total dérivé ; clé étrangère vers **Asset**.

PriceHistory Série temporelle de clôtures simulées ou issues d'une évolution de marché ; contrainte d'unicité (`AssetId, Date`).

PortfolioBudget Budget initial et liquidité disponible (mise à jour lors des opérations).

PortfolioValuePoint Valeur totale du portefeuille (cash + positions au cours du jour) pour une date donnée ; index unique sur la date.

4.2 Relations et intégrité

Les relations `Asset` $\rightarrow$ `Transaction` et `Asset` $\rightarrow$ `PriceHistory` sont en **one-to-many** avec suppression en cascade côté dépendant, configurées dans `OnModelCreating`. L'index unique sur `Asset.Symbol` garantit l'absence de doublons de tickers.

4.3 Migrations

Les migrations EF Core sont stockées dans `GestionPortefeuille.Infrastructure/Migrations/`, notamment :

- création initiale des tables (`InitialCreate`) ;
- ajout de `PortfolioValuePoints` (`AddPortfolioValuePoint`).

Au démarrage de l'application, `Database.Migrate()` applique automatiquement les migrations en attente, ce qui facilite la démonstration sur une machine neuve. Un *seed* insère un budget par défaut et des actifs de démonstration si la base est vide.

4.4 Choix SQLite

SQLite convient à un projet étudiant : fichier unique, pas de serveur à installer, sauvegarde du dépôt possible en excluant `*.db` via `.gitignore`. Pour une mise en production, une base serveur (PostgreSQL, SQL Server) serait préférable (concurrence, sauvegardes, taille).

Chapitre 5

Réalisation technique

5.1 Composition racine et pipeline

Le fichier `Program.cs` de l'application Web enregistre Blazor (composants interactifs serveur), le `DbContext` `SQLite`, et les services métier via une méthode d'extension :

Listing 5.1 – Extrait typique d'enregistrement des services

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("
        DefaultConnection")));
builder.Services.AddPortfolioServices(builder.Configuration);
```

Après construction du pipeline (fichiers statiques, antiforgery, Razor Components), une portée `scoped` exécute les migrations et le *seed*.

5.2 Services métier

5.2.1 Portefeuille (`PortfolioService`)

Calcule l'instantané `PortfolioSnapshot` à partir des actifs, des transactions et du budget. Le **coût moyen pondéré** (CMP) des titres encore détenus est obtenu par une passe chronologique sur les opérations (`PositionCostHelper`) : à l'achat, le CMP est recalculé ; à la vente partielle, le CMP unitaire des titres restants est conservé (logique courante pour une valorisation de portefeuille pédagogique).

5.2.2 Transactions (`TransactionService`)

Vérifie la liquidité pour les achats et la quantité détenue pour les ventes, met à jour `PortfolioBudget.AvailableCash`, persiste la transaction, puis enregistre un point d'historique de valeur totale via `IPortfolioService`.

5.2.3 Actifs (`AssetService`)

CRUD, filtrage LINQ (recherche insensible à la casse sur symbole/nom, filtre par type), pagination serveur (`PagedResult<T>`).

5.2.4 Prix marché (`PriceDataService`)

Selon `PriceData.Mode` (`Simulated`, `Api`, `None`), met à jour ou non les `CurrentPrice`. En mode API, utilisation de `Finnhub` (actions, clé API) et `CoinGecko` (cryptos mappées), avec mise en cache mémoire pour limiter les appels.

5.2.5 Alertes (`PortfolioAlertService`)

Agrège instantané et analytique pour produire une liste de `PortfolioAlert` selon les seuils `appsettings.json`.

5.3 Validation

Les entités `Asset`, `Transaction`, `PortfolioBudget`, etc. portent des attributs `[Required]`, `[Range]`, `[StringLength]`. Les formulaires Blazor utilisent `EditForm`, `DataAnnotationsValidator` et `ValidationSummary`.

5.4 Asynchronisme

Toutes les opérations d'accès base dans les services sont asynchrones (`Task`, `async/await`), ce qui évite de bloquer les threads du pool lors des E/S réseau et base de données.

Chapitre 6

Extension analytique et data science

6.1 Données d'entrée

L'analytique s'appuie sur `PriceHistory` : si aucun historique n'existe pour un actif, une série de prix est **simulée** (marche aléatoire bornée, graine dérivée du symbole) sur une fenêtre de jours configurable, afin de garantir des graphiques et des métriques en démonstration sans dépendre d'Internet.

6.2 Moyenne mobile simple (SMA)

Pour une fenêtre de N jours (ex. $N = 20$), la SMA au jour t est la moyenne arithmétique des N derniers prix de clôture :

$$\text{SMA}_N(t) = \frac{1}{N} \sum_{i=0}^{N-1} P_{t-i}. \quad (6.1)$$

Un **signal de tendance** grossier compare le prix courant à la SMA (écart en pourcentage) : au-dessus (hausse), en dessous (baisse), zone neutre autour de zéro.

6.3 Volatilité

Les rendements journaliers sont définis par $r_t = \frac{P_t - P_{t-1}}{P_{t-1}}$. L'écart-type empirique $\hat{\sigma}$ des r_t sur l'échantillon disponible est calculé (estimateur non biaisé avec dénominateur $n - 1$). Une **volatilité annualisée approximative** est obtenue par

$$\sigma_{\text{ann}} \approx \hat{\sigma} \times \sqrt{252}, \quad (6.2)$$

en supposant 252 jours de négociation par an (hypothèse standard en finance, ici à visée pédagogique).

6.4 Ratio de Sharpe simplifié

Sans taux sans risque explicite ($r_f = 0$), un indicateur *Sharpe-like* annualisé est :

$$S \approx \frac{\bar{r}}{\hat{\sigma}} \times \sqrt{252}, \quad (6.3)$$

où $\bar{r}$ est la moyenne des rendements journaliers. Ce rapport mesure le rendement moyen par unité de risque (écart-type) ; il doit être interprété avec prudence sur des séries courtes ou simulées.

6.5 Drawdown maximal

Sur la série de prix (P_t), on suit le *peak* courant et l'écart relatif au peak. Le drawdown au temps t est $\frac{P_t - \text{peak}_t}{\text{peak}_t}$. Le **maximum drawdown** est le minimum (le plus négatif) de ces rapports, exprimé en valeur absolue en pourcentage pour l'affichage.

6.6 Backtest de règle SMA

À partir du jour où la SMA est disponible, une stratégie simplifiée :

- si le prix dépasse la SMA et qu'aucune position n'est détenue, investir la totalité du cash au prix du jour ;
- si le prix passe sous la SMA et qu'une position est ouverte, tout liquider au prix du jour.

Le résultat est comparé à un **buy-and-hold** : investissement intégral au premier jour de la fenêtre utile, conservation jusqu'à la fin. Le nombre d'opérations de la stratégie est comptabilisé. Ce backtest illustre l'impact d'une règle technique sur un historique donné (sans frais, sans slippage — limites à mentionner à l'oral).

6.7 Intérêt pour la note « créativité »

Ces éléments relient explicitement le projet à des notions de **statistique descriptive**, de **mesure du risque** et de **simulation décisionnelle**, tout en restant dans le périmètre du cours (.NET, LINQ, affichage Blazor).

Chapitre 7

Interface utilisateur (Blazor)

7.1 Structure de navigation

L'application propose un menu latéral avec les entrées : tableau de bord, actifs, transactions, budget, outils (marchés, import CSV, backtest). Le layout principal inclut un commutateur de **thème clair/sombre** (attribut `data-bs-theme` et stockage `localStorage`).

7.2 Pages principales

Tableau de bord Cartes KPI (valeur totale, liquidité, gain/perte, ROI), alertes, graphiques Chart.js (répartition en anneau, courbe d'historique de valeur), répartition par actif et par type, bloc analytique par symbole, tableau de performance avec CMP et coût de revient.

Actifs Formulaire CRUD, filtres, pagination, tri, recherche avec *debounce*.

Transactions Saisie avec validation, résumé budget, historique paginé, tri, export CSV global.

Budget Ajustement du budget initial et de la liquidité.

Outils Documentation des modes de prix, bouton de rafraîchissement API, zone de texte pour import CSV, formulaire de backtest.

7.3 Technologies front

- **Bootstrap** (fichiers locaux sous `wwwroot/lib/bootstrap`) pour la grille et les composants.
- **Chart.js** chargé via CDN dans `App.razor`, scripts personnalisés `portfolio-charts.js` et `theme.js`.
- **Interop JavaScript** (`IJSRuntime`) depuis les composants interactifs pour initialiser et mettre à jour les graphiques.

7.4 Mode de rendu

Les pages à interactions fortes utilisent `@rendermode InteractiveServer`. Le reconnect modal Blazor est conservé pour une meilleure expérience en cas de perte de circuit SignalR.

Chapitre 8

Validation et déploiement local

8.1 Vérifications automatisées

La commande `dotnet build` sur la solution ou le projet Web doit s'exécuter sans erreur. Les migrations sont appliquées au lancement ; en cas de doute, `dotnet ef database update` peut être exécuté avec le projet d'infrastructure et le projet Web comme startup.

8.2 Vérifications manuelles recommandées

1. Lancer l'application (`dotnet run` ou `dotnet watch` depuis `GestionPortefeuille.Web`).
2. Parcourir le CRUD actifs : création, édition, suppression, filtres.
3. Enregistrer des transactions achat/vente : vérifier la diminution/augmentation de la liquidité et le refus des opérations invalides.
4. Consulter le tableau de bord : cohérence des pourcentages de répartition et des gains/pertes avec le CMP.
5. Onglet Outils : import CSV de prix, backtest sur un actif disposant d'historique.
6. Sous Windows : si `dotnet build` échoue sur des fichiers verrouillés (`.dll`, `.pdb`), arrêter toute instance en cours (`Ctrl+C`) ou terminer le processus hôte avant de recompiler.

8.3 Limites connues

- Historique de prix **simulé** : les conclusions du backtest et de la volatilité sont illustratives.
- APIs marché soumises à quotas, clés et disponibilité réseau ; le mapping crypto est limité à une liste de symboles.
- Pas d'authentification multi-utilisateur dans la version décrite (évolution possible).

Chapitre 9

Conclusion et perspectives

9.1 Bilan

Le projet livre une application Web **complète** répondant aux exigences du module : Blazor, EF Core Code First avec migrations, architecture en couches, CRUD et recherche LINQ, tableau de bord avec graphiques, validation par annotations, code asynchrone. L'extension **data science** renforce la dimension analytique (risque, tendance, backtest) et supporte la partie « créativité » du barème.

9.2 Pistes d'évolution

- **Authentification** (ASP.NET Core Identity) et portefeuilles utilisateur isolés.
- **Tests unitaires** sur `PositionCostHelper`, règles de transaction et calculs d'analytique.
- **CI/CD** (GitHub Actions) : build + tests + publication.
- **API REST** optionnelle pour alimenter une mobile app ou un tableau externe.
- **Coût moyen** plus avancé (FIFO réel) et frais de transaction dans le modèle.
- Base **PostgreSQL** ou **SQL Server** pour un déploiement multi-utilisateurs.

9.3 Remerciements / répartition travail d'équipe

[À compléter : rôles des membres, répartition des tâches, difficultés rencontrées.]

Annexe A

Annexe A — Extraits de code significatifs

A.1 Point d'entrée et migrations (Program.cs)

Listing A.1 – GestionPortefeuille.Web/Program.cs (structure principale)

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddRazorComponents()
    .AddInteractiveServerComponents();
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("
        DefaultConnection")));
builder.Services.AddPortfolioServices(builder.Configuration);

var app = builder.Build();
// ... pipeline HTTP (fichiers statiques, antiforgery, Razor Components)
...

using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    db.Database.Migrate();
    await DbInitializer.SeedAsync(db);
}

app.Run();
```

A.2 Injection de dépendances (AddPortfolioServices)

Listing A.2 – GestionPortefeuille.Services/ServiceCollectionExtensions.cs

```
public static IServiceCollection AddPortfolioServices(
    this IServiceCollection services, IConfiguration configuration)
{
    services.Configure<AlertOptions>(configuration.GetSection(
        AlertOptions.SectionName));
    services.Configure<PriceDataOptions>(configuration.GetSection(
        PriceDataOptions.SectionName));
}
```

```
services.AddMemoryCache();
services.AddHttpClient<IPriceDataService, PriceDataService>(client
=>
{
    client.DefaultRequestHeaders.UserAgent.ParseAdd("
    GestionPortefeuille/1.0 (demo)");
    client.Timeout = TimeSpan.FromSeconds(20);
});

services.AddScoped<IAssetService, AssetService>();
services.AddScoped<ITransactionService, TransactionService>();
services.AddScoped<IPortfolioService, PortfolioService>();
services.AddScoped<IAnalyticsService, AnalyticsService>();
services.AddScoped<IPortfolioAlertService, PortfolioAlertService>();
return services;
}
```

A.3 Lecture complémentaire

Les entités et interfaces se trouvent dans `GestionPortefeuille.Core`, le `DbContext` et les migrations dans `GestionPortefeuille.Infrastructure`, les pages Blazor sous `GestionPortefeuille.Web/Components`. Un diagramme de classes Mermaid est fourni à la racine du dépôt dans `DiagrammeClasses.md`.

Annexe B

Annexe B — Commandes utiles (.NET CLI)

B.1 Restaurer et compiler

```
cd "chemin\vers\GestionPortefeuille2"
dotnet restore
dotnet build GestionPortefeuille.sln
```

B.2 Exécuter l'application Web

```
cd GestionPortefeuille.Web
dotnet run
```

Mode développement avec rechargement à chaud :

```
dotnet watch run
```

B.3 Migrations Entity Framework Core

Depuis la racine de la solution (adapter les chemins si besoin) :

```
dotnet ef migrations add NomMigration ^
--project GestionPortefeuille.Infrastructure ^
--startup-project GestionPortefeuille.Web
```

```
dotnet ef database update ^
--project GestionPortefeuille.Infrastructure ^
--startup-project GestionPortefeuille.Web
```

Remarque : sous PowerShell, remplacer ^ par le caractère d'échappement de continuation de ligne adéquat ou écrire la commande sur une seule ligne.

B.4 Problème de fichiers verrouillés (Windows)

Si le build échoue avec un message indiquant qu'un .dll ou .pdb est utilisé par un autre processus :

1. Arrêter l'application (Ctrl+C dans le terminal où `dotnet watch` ou `dotnet run` tourne).
2. Si nécessaire : `dotnet build-server shutdown`, puis relancer `dotnet build`.

B.5 Compiler le PDF du rapport

```
cd Rapport
pdflatex -interaction=nonstopmode rapport.tex
pdflatex -interaction=nonstopmode rapport.tex
```

Ou avec latexmk :

```
latexmk -pdf -interaction=nonstopmode rapport.tex
```